

Analysis: Doubly-sorted Grid

A counting problem, with a board size not too big. The problem gives a quick impression of dynamic programming on a space of exponentially many states. And it is indeed so.

By the limits of this problem, let us say the size is the larger of m and n . A solution with $2^{2 \cdot \text{size}}$ states is fine, while a solution with $2^{4 \cdot \text{size}}$ states is probably only good for the small dataset. However, for regular programming contest goers, there are many conventional ways to define the states for similar grid problems that fall into the latter category.

So, the key part of the problem is to find the right state space. Once it is found, the finalists can no doubt carry out the dynamic programming solution easily.

The basic picture is the lattice paths. Specifically, let us consider, in a doubly sorted grid, all the letters less than or equal to a particular character. They form an upwards closed region towards the top-left corner. In other words, if the letter in (r, c) is no greater than the prescribed character, so is the letter in (r', c') , if $r' \leq r$ and $c' \leq c$. As a result, the boundary separating this region and the rest of the grid forms a lattice path from the bottom-left to the top-right, and can only go north or east. This is a well known subject -- there are $\binom{m+n}{m}$ such paths in total. Let us call them *monotone paths*. For two monotone paths, we say one dominates the other if one never goes above the other. Any doubly sorted grid corresponds in a one to one fashion to 26 monotone paths (some of which may be identical), one for each letter, and the path for a bigger letter dominates the paths for the smaller letters. The left picture below depicts the situation when there are three letters; and the monotone boundaries for 'a' and 'b' are highlighted.



Just one step further. Let us focus not only the exact boundary for a letter but any monotone path. For any monotone path P and any letter c , define

$\text{dp}[P][c] :=$ the number of ways one can fill all the squares above the path P , using only the letters no greater than c , so that the upper part is doubly sorted, and any pre-filled letter in the upper part is respected.

For any monotone path except the most dominated one, we have one or more *maximal* points, those are the points where the path goes east then followed by a step upwards. In the second picture above, we highlight a monotone path with its maximal points colored. To compute $\text{dp}[P][c]$, we can divide the situation into two cases. (1) The letter c does not appear at all. There are $\text{dp}[P][c-1]$ ways to do so. (2) Otherwise, c must appear in at least one of the maximal points of P . For each non-empty subset of the maximal points, we can assign the letter c to them, reducing our task to $\text{dp}[P'][c]$, where P' is a path that only differs from P in that subset of maximal points. We use inclusion-exclusion formula on all the non-empty subsets to compute the contribution to $\text{dp}[P][c]$ in this case.

Such a solution is relatively intuitive, and is fast enough under our constraints. By adding one more helper, one can find a faster solution. Now let us refine

$dp[P][c][k]$:= the number of ways one can fill all the squares above the path P , using only the letters no greater than c , *and the letter c does not occur anywhere after column k* , so that the upper part is doubly sorted, and any pre-filled letter in the upper part is respected.

We leave the implementation details as an easy exercise for interested readers. We mention that, when $m=n$, the number of states is $26 \cdot \binom{2n}{n} \cdot n = \Theta(4^n n^{0.5})$. Although the computation of a single $dp[P][c][k]$ may involve up to n steps, the running time can be shown as $\Theta(4^n n^{0.5})$ by a simple amortized analysis -- for fixed P and c , we need $O(n)$ steps in total to compute the table for all k .

Analysis: Lights

There are various ways to solve this problem. The solutions can be split into two kinds of approaches. One way is to solve the problem exactly (or, to arbitrary precision), the other is to approximate the answer with high enough precision. The exact solutions generally try to divide the square into subareas of the same color, compute the area of each subarea separately, and add up the totals for each color. We discuss these first.

Intersection of (mostly) triangles

First, consider just one light. We want to compute the total area illuminated by the light. To do that, compute the tangent lines from the light to each pillar, and lines from the light to each corner of the room, and consider each "cone" between two adjacent lines separately. Each cone will either end up hitting a wall, or hitting a pillar. In the first case we get a triangle, whose area we can easily compute. In the second case we get a "quasi-triangle", that is, a triangle minus a disk segment. Here, we need to subtract the area of the disk segment. We can compute the area of a disk segment by subtracting a triangle from the area of a disk sector (a "pie slice").

Once we have the total area covered by each light, we need one more thing: the area covered by both lights. We can take each pair of triangles or quasi-triangles generated in the previous step, and compute the common area between them. Now we need to compute the area of the intersection of two triangles or quasi-triangles.

A simple way to approach this is to first treat quasi-triangles as triangles (include the disk segment). Now we compute the intersection between the two triangles, which gives us a polygon (up to six sides). If one or both of the triangles were actual triangles, or when the pillars subtracted from the two quasi-triangles were different, the polygon is the correct answer - there is no need to account for the subtracted disk segments, because each segment is outside the other triangle anyway.

The only nasty case comes up when we have two quasi-triangles ending at the same pillar. In that case, we first compute the intersection polygon, and then we subtract the pillar from the polygon. To do that, remove those edges and parts of edges of the polygon that fall inside the circle and replace them with one edge. The answer will be the area of the reduced polygon, again minus a disk sector cut off by a line, which we already know how to compute.

Line sweeping

Line sweeping is a common technique in computational geometry. We sweep a vertical line from the left edge to the right. As in the solution above, the interesting rays are the tangent rays from lights to circles. The interesting moments are when the x-coordinate of the vertical line reaches one of the following. (1) A light. (2) A pillar starts or ends. (3) An interesting ray touches or intersects a circle, or hits the wall. (4) Two interesting rays intersect.

Now, let $x_1 < x_2$ be two adjacent interesting moments. The vertical strip between x_1 and x_2 is divided into pieces. Each piece is bounded above and below by a general segment -- a line segment or an arc. By the definition of the interesting moments, nothing interesting will happen in the middle, and each piece is of one color. So we can sample an arbitrary point from each piece to decide the color. The pieces are not convex, but this is not a problem -- they are convex on any vertical line so we can easily find a point that is inside each piece. The area of each piece is also easy to compute -- it is basically a trapezoid, possibly degenerating into a triangle if

the upper and lower boundaries meet at one end, and one needs to subtract a disk segment for each arc-boundary.

Line sweeping is often used with nice data structures to achieve good complexity. But that is not our primary concern here. We used it for the simplicity of the implementation -- the only geometric operations needed here are intersections between lines and circles.

Approximations

The problem requires a relative or absolute error of at most 10^{-5} , while the total room area is 10000. Cases requiring the most care are those when one of the four colors has an area less than 1, in which case the error we can make relative to the area of the whole room is 10^{-9} .

The simplest approach would be to sample a lot of points either randomly, or in a regular grid, compute the color of each sample and assume that the sample is representative of the correct answer. The above error estimation suggests though that to get enough precision, we would need to sample on the order of 10^9 points (or more, due to random deviations). This is too much for a solution to run within the time limit. A smarter approach is needed.

Computing the area can be seen as a problem of computing a two-dimensional integral. A hybrid approach is also possible: we can see it as a one-dimensional integral along the x coordinate, and for each x coordinate we sample we can compute the exact answer by looking at which segment of the vertical line is in what color. This one-dimensional sub-problem is somewhat simpler to do than solving the full two-dimensional problem exactly.

In either case, whether we compute a two-dimensional integral or just a one-dimensional one for a more complex function, we need a smart way to approximate the integral. Uniform or random sampling is not enough.

You can search the web for methods of [numerical integration](#). In this problem, an adaptive algorithm is needed, which means that we sample more "interesting" areas with more samples than the less "interesting" ones. "Interesting" can be defined as "large changes in values" (large first derivative) or "wild changes in values" (large second derivative).

One simple algorithm is to write the integration procedure as a recursive function. We recursively try splitting the interval into smaller ones, and see how much the answer changes through such increases of precision. We stop the recursion when the answer changes very little, which means the interval is small enough or the function is smooth enough in the interval. This will result in sampling the more "interesting" areas more accurately.

Analysis: Marbles

Last year we were trying to solve different instances of this problem. It took a long time to converge to this particular shape, and even after we settled on the current requirements, we were still tweaking the input limits at 2am the night before the contest to make the problem a bit more interesting. The problem nicely combines together dynamic programming, greedy and graph related notions like biconnected components and trees.

The first step is deciding if a particular configuration is or isn't solvable. If for two colors their corresponding marbles alternate it means that the two pairs of marbles need to be joined by curves on opposite sides of the horizontal line $Y=0$. We can build a graph where the nodes represent pairs of same-color marbles and form a graph with edges between pairs of marbles that alternate. We can draw the paths with no intersection if and only if this graph is bipartite.

Next, for solvable configurations we compute the minimum height. The pairs graph can have many connected components, and for each such component we can choose two ways of drawing the lines (with the first pair of marbles above the $Y=0$ line or below it), so in total we would have $O(2^{\text{components}})$ configurations. This idea can solve the small case but is too time consuming for the large case.

Solving the large case requires us to use dynamic programming. Our state will be defined by *left*, *right*, *height_up* and *height_down*. For each state we compute a boolean value which tells us if the subproblem which uses the set of marbles with indexes from *left* to *right* can be solved in the vertical range $[-\text{height_down} .. \text{height_up}]$. Computing this value is a bit tricky, what we need to notice is that we can try each of the two ways of drawing the component that starts at index *left*. Then an important observation is that we can draw each path with the maximum height possible if the line is above the X axis or maximum depth possible if the line is below the X axis as long as our drawing is within the $[-\text{height_down} .. \text{height_up}]$ vertical range. Using these ideas we can come up with an $O(n^5)$ algorithm.

We can improve on this solution by using the state (*left*, *right*, *height_up*) and for each state finding the smallest *height_down* for which the subproblem $[\text{left} .. \text{right}]$ is solvable. Now we notice that we should use dynamic programming on pairs of *left* and *right* where connected components of marbles start and finish. This will make *right* uniquely defined by *left*. Thus we have reduced the state space to $O(n^2)$ states. We also notice that the connected components form a tree-like structure where we need to solve the innermost components first and then solve outer components, much like visiting the leaves of a tree first and getting closer and closer to the root. Now each connected component will be analyzed just once at an upper component level so the overall algorithm will take $O(n^2)$ time.

Here's Tomek Czajka's solution:

```
#include <algorithm>
#include <cassert>
#include <cstdio>
#include <map>
#include <string>
#include <vector>
using namespace std;
#define REP(i,n) for(int i=0;i<(n);++i)
template<class T> inline int size(const T&c) { return c.size(); }
const int INF = 1000000000;
```

```

int n; // number of types of marbles
vector<vector<int> > where; // [n][2]
vector<int> marbles; // [2*n]

void readInput() {
    char buf[30];
    map<string,int> dict;
    scanf("%d", &n);
    marbles.clear(); marbles.reserve(2*n);
    where.clear(); where.resize(n);
    for(int i=0;i<2*n;++i) {
        scanf("%s",buf);
        string s = buf;
        map<string,int>::iterator it = dict.find(s);
        int m;
        if(it==dict.end()) {
            m = size(dict);
            dict[s] = m;
        } else {
            m = it->second;
        }
        marbles.push_back(m);
        where[m].push_back(i);
    }
}

struct Event {
    int x,t;
    // t=0 start top, 1 end top
    // t=2 start bot, 3 end bot
};

vector<int> vis;

bool cross(int m1,int m2) {
    return
        where[m1][0] < where[m2][0] &&
        where[m2][0] < where[m1][1] &&
        where[m1][1] < where[m2][1] ||
        where[m2][0] < where[m1][0] &&
        where[m1][0] < where[m2][1] &&
        where[m2][1] < where[m1][1];
}

void dfs(int m,int sign) {
    if(vis[m]==sign) return;
    if(vis[m]==-sign) throw 0;
    vis[m]=sign;
    REP(i,n) if(i!=m && cross(m,i)) dfs(i,-sign);
}

vector<vector<Event> > cacheCalcEvents;

const vector<Event> &calcEvents(int startx) {
    vector<Event> &res = cacheCalcEvents[startx];
    if(!res.empty()) return res;

```

```

vis.assign(n,0);
dfs(marbles[startx],1);
REP(x,2*n) {
    int m = marbles[x];
    if(vis[m]==0) continue;
    int nr=0;
    if(where[m][nr] != x) ++nr;
    assert(where[m][nr]==x);
    Event e; e.x=x;
    e.t = (1-vis[m]) + nr;
    res.push_back(e);
}
return res;
}

vector<vector<int> > cacheCalcH2;

int calcH2(int a,int b,int h1) {
    if(h1<0) return INF;
    if(a==b) return 0;
    int &res = cacheCalcH2[a][h1];
    if(res!=-1) return res;
    const vector<Event> &events = calcEvents(a);
    res = INF;
    for(int mask = 0; mask<=2; mask+=2) {
        int top=0, bot=0;
        int h2 = 0;
        REP(i,size(events)+1) {
            int alpha = i==0 ? a : events[i-1].x + 1;
            int beta = i==size(events) ? b : events[i].x;
            h2 = max(h2, calcH2(alpha, beta, h1 - top) + bot);
            if(i!=size(events)) {
                switch(events[i].t ^ mask) {
                    case 0: ++top; break;
                    case 1: --top; break;
                    case 2: ++bot; break;
                    case 3: --bot; break;
                }
            }
        }
        res = min(res, h2);
    }
    return res;
}

int solve() {
    int res = INF;
    cacheCalcH2.assign(2*n, vector<int>(n+1,-1));
    cacheCalcEvents.clear(); cacheCalcEvents.resize(2*n);
    try {
        REP(h1,n+1) {
            res = min(res, h1 + calcH2(0,2*n,h1));
        }
        return res;
    } catch(int) { return INF; }
}

```

```
int main() {
    int ntc; scanf("%d", &ntc);
    REP(tc,ntc) {
        readInput();
        int res = solve();
        if(res==INF) res = -1;
        printf("Case #%d: %d\n", tc+1, res);
    }
}
```


Analysis: Min Perimeter

This problem is similar to the classical problem of finding the closest pair in a set of points. Algorithms that solve the closest-pair problem can be adapted to solve this one.

The number of points can be pretty large so we need an efficient algorithm. We can solve the problem in $O(n \log n)$ time using divide and conquer. We will split the set of points by a vertical line into two sets of equal size. There are now three cases for the minimum-perimeter triangle: its three points can either be entirely in the left set, entirely in the right set, or it can use points from each half.

We find the minimum perimeters for the left and right sets using recursion. Let the smallest of those perimeters be p . We can use p to make finding the minimum perimeter of the third case efficient, by only considering triangles that could have an area less than p .

To find the minimum perimeter for the third case (if it is less than p) we select the points that are within a distance of $p/2$ from the vertical separation line. Then we iterate through those points from top to bottom, and maintain a list of all the points in a box of size $p \times p/2$, with the bottom edge of the box at the most recently considered point. As we add each point to the box, we compute the perimeter of all triangles using that point and each pair of points in the box. (We could exclude triangles entirely to the left or right of the dividing line, since those have already been considered.)

We can prove that the number of points in the box is at most 16, so we only need to consider at most a small constant number of triangles for each point.

Splitting the current set of points by a vertical line requires the points to be sorted by x and going through the points vertically requires having the points sorted by y . If we do the y sort at each step that gives us an $O(n \log^2 n)$ algorithm, but we can keep the set of points twice, one array would have the points sorted by x and one would have the points sorted by y , and this way we have an $O(n \log n)$ algorithm.

The time limits were a bit tight and input limits were large because some $O(n^2)$ algorithms work really well on random cases. This is why during the contest some solutions that had the right idea but used a $p \times p$ box size or sorted by y at each step didn't manage to solve the large test cases fast enough.

You can read Tomek Czajka's source to get the details of a good implementation:

```
#include <algorithm>
#include <cassert>
#include <cmath>
#include <cstdio>
#include <cstdlib>
#include <vector>
using namespace std;
#define REP(i,n) for(int i=0;i<(n);++i)
template<class T> inline int size(const T&c) { return c.size(); }

const int BILLION = 1000000000;
const double INF = 1e20;
typedef long long LL;
```

```

struct Point {
    int x,y;
    Point() {}
    Point(int x,int y):x(x),y(y) {}
};

inline Point middle(const Point &a, const Point &b) {
    return Point((a.x+b.x)/2, (a.y+b.y)/2);
}

struct CmpX {
    inline bool operator()(const Point &a, const Point &b) {
        if(a.x != b.x) return a.x < b.x;
        return a.y < b.y;
    }
} cmpx;

struct CmpY {
    inline bool operator()(const Point &a, const Point &b) {
        if(a.y != b.y) return a.y < b.y;
        return a.x < b.x;
    }
} cmpy;

inline LL sqr(int x) { return LL(x) * LL(x); }

inline double dist(const Point &a, const Point &b) {
    return sqrt(double(sqr(a.x-b.x) + sqr(a.y-b.y)));
}

inline double perimeter(const Point &a,
                        const Point &b,
                        const Point &c) {
    return dist(a,b) + dist(b,c) + dist(c,a);
}

double calc(int n, const Point points[],
            const vector<Point> &pointsByY) {
    if(n<3) return INF;
    int left = n/2;
    int right = n-left;
    Point split = middle(points[left-1], points[left]);
    vector<Point> pointsByYLeft, pointsByYRight;
    pointsByYLeft.reserve(left);
    pointsByYRight.reserve(right);
    REP(i,n) {
        if(cmpx(pointsByY[i], split))
            pointsByYLeft.push_back(pointsByY[i]);
        else
            pointsByYRight.push_back(pointsByY[i]);
    }
    double res = INF;
    res = min(res, calc(left, points, pointsByYLeft));
    res = min(res, calc(right, points+left, pointsByYRight));
    static vector<Point> closeToTheLine;
    int margin = (res > INF/2) ? 2*BILLION : int(res/2);
    closeToTheLine.clear();
}

```

```

closeToTheLine.reserve(n);
int start = 0;
for(int i=0;i<n;++i) {
    Point p = pointsByY[i];
    if(abs(p.x - split.x) > margin) continue;
    while(start < size(closeToTheLine) &&
           p.y - closeToTheLine[start].y > margin) ++start;
    for(int i=start;i<size(closeToTheLine);++i) {
        for(int j=i+1;j<size(closeToTheLine);++j) {
            res = min(res, perimeter(p, closeToTheLine[i],
                                     closeToTheLine[j]));
        }
    }
    closeToTheLine.push_back(p);
}
return res;
}

double calc(vector<Point> &points) {
    sort(points.begin(), points.end(), cmpx);
    vector<Point> pointsByY = points;
    sort(pointsByY.begin(), pointsByY.end(), cmpy);
    return calc(size(points), &points[0], pointsByY);
}

int main() {
    assert(0==system("cat > Input.java"));
    fprintf(stderr, "Compiling generator\n");
    assert(0==system("javac Input.java"));
    fprintf(stderr, "Running generator\n");
    assert(0==system("java -Xmx512M Input > input.tmp"));
    fprintf(stderr, "Solving\n");
    FILE *f = fopen("input.tmp", "r");
    int ntc; fscanf(f, "%d", &ntc);
    REP(tc, ntc) {
        int n; fscanf(f, "%d", &n);
        vector<Point> points;
        points.reserve(n);
        REP(i, n) {
            int x, y; fscanf(f, "%d%d", &x, &y);
            points.push_back(Point(2*x-BILLION, 2*y-BILLION));
        }
        double res = calc(points);
        printf("Case #d: %.15e\n", tc+1, res/2);
    }
    fclose(f);
}

```

Analysis: Wi-fi Towers

Connection graph

We start by representing the problem as a graph problem. Each tower is a vertex in the graph and has a weight equal to its score. If a tower A has another tower B in its range, we represent this fact as a directed edge from A to B. The problem is to choose a set of vertices with maximum weight such that for every edge from A to B, if A is chosen then B is also chosen. In the following, we assume the number of towers (vertices) is V and the number of edges is E .

Reduce to an instance of MIN-CUT

To reduce the problem to an instance of MIN-CUT, we create a flow network as follows. Create a source vertex, a sink vertex, and one vertex for each tower. Suppose a tower has score s . If $s > 0$, create an edge from the vertex to the sink with capacity s . If $s < 0$, create an edge from the source to this vertex with capacity $|s|$. Finally, for every edge in the connection graph, create a similar edge in the flow network with infinite capacity. The network has $V + 2 = O(V)$ vertices and $O(V + E)$ edges.

Now every finite cut in the graph represents a choice of towers - we choose every tower on the same side of the cut as the source. The infinite capacity edges enforce that the choice follows the given constraints (otherwise we get a cut of infinite weight). The edges from the source and to the sink penalize the solution appropriately for choosing towers with negative scores and for not choosing towers with positive scores. If the value of the best cut is C , the answer is $S - C$, where S is the sum of positive tower scores.

Solving MIN-CUT

By the max-flow min-cut theorem, we can solve the MIN-CUT instance generated above by computing the maximum flow in the same graph. To compute the maximum flow, we can use the Edmonds-Karp algorithm (a variant of Ford-Fulkerson which selects augmenting paths using BFS), which results in complexity bounded by $O(V(V+E)^2) = O(V^5)$. In practice, this was fast enough to solve all possible test cases.

One could also use a more complicated push-relabel max-flow algorithm which, with a FIFO vertex selection rule, results in complexity $O(V^3)$.

Yet another algorithm, thanks to integral capacities, is the capacity scaling variant of Ford-Fulkerson (start by searching for augmenting paths with weights being large powers of 2 first, and then decrease). This results in $O((V+E)^2 \log F) = O(V^4 \log F)$ where F is the maximum value of the flow.

Reducing the number of edges

Finally, there is a geometric trick using which we can reduce the number of edges E to $O(V)$, thus reducing the complexity of the algorithms above. The complexity of Edmonds-Karp becomes $O(V^3)$ and of capacity scaling: $O(V^2 \log F)$.

First, notice that we can remove edges without changing the final answer as long as the transitive closure of the graph stays the same.

The crucial trick is to see that if there are two directed edges A-C and B-C, and the angle ACB is smaller than 60 degrees, then we can remove the longer edge. Suppose A-C is longer than B-C. Then if we remove the edge A-C, there is still going to be an indirect connection A-B-C (using shorter or equal length edges), thanks to the fact that the range of A is a circle.

If we keep doing this, every vertex will end up with at most 6 incoming edges, thus reducing the total number of edges to at most $6V$.

More information

This problem is also equivalent to the [Minimum Closure Problem](#), which was studied in the 1970s and has applications in the mining industry.

[Max Flow Min Cut Theorem](#)

Analysis: Year of More Code Jam

The setting of this problem is no doubt discrete probability. From the definition, the space consists of N^T equally likely possible outcomes. That can be, under our limits, as huge as 10^{450} . Clearly, a naive approach is not feasible.

But let us do a little exercise in probability. Define the random variable X_i to be the number of contests on the i -th day, the quantity we want to compute is the average of X_i^2 , i.e., the expectation $E(\sum X_i^2)$. By the linearity of the expectation, we have

$$E(\sum_{1 \leq i \leq N} X_i^2) = \sum_{1 \leq i \leq N} E(X_i^2).$$

So let us focus on the computation of the variable for a fixed day for the moment. Pick any i , and let $X := X_i$. Let us define more random variables. Define Y_j to be the *indicator* of whether the j -th tournament will have a contest on the i -th day. Clearly, $X = \sum_j Y_j$. So,

$$(*) \quad E(X^2) = E((\sum_{1 \leq j \leq T} Y_j)^2) = \sum_{1 \leq j \leq T} E(Y_j^2) + 2 \sum_{1 \leq j < k \leq T} E(Y_j Y_k).$$

We observe that each terms in the last expression is easy to compute. Being the indicator random variables, the Y 's take value 0 or 1. So

- Y_j^2 always has the same value as Y_j , and its expectation is just the probability that Y_j is 1, i.e., tournament j has a contest on day i .
- $Y_j Y_k$ is 1 if and only if both Y_j and Y_k are 1. The expectation is the probability that both the j -th and the k -th tournament has a contest on day i .

Let the input for the tournament j , i.e., the contest pattern be $d_1=1, d_2, \dots, d_m$. Denote $D(i,j)$ be the number of the d 's for tournament j that are less than or equal to i . There are N choices for the starting date of a particular tournament. It is easy to see that the first probability above is $D(i,j) / N$; while the second probability is $D(i,j)D(i,k) / N^2$.

So far we addressed the problem just for a single day i . We need to do this for every i . There are 10^9 of them. But notice that, as long as there is no input $d_t = i$, $D(i-1, j) = D(i, j)$ for all j . This means that the expectation for the i -th day is the same as the expectation for the $(i-1)$ -th. There are at most $T \max(M) \leq 2500$ such d 's in the input, so we need to compute $(*)$ for at most 2500 days. For our problem, it is good enough to realize that there is no $d > 10000$. So all the expectations after the 10000-th day are the same. We can just do the computation for the first 10000 days, and for the rest, a simple multiplication.

The last problem is the need for big integers. At the first glance we might have both numerators and denominators as big as 10^{450} . But that is not the truth. Simply observe the above answer, which is a sum of various $D(i,j) / N$ and $D(i,j)D(i,k) / N^2$. We actually proved that the denominator is never bigger than N^2 . A careful implementation with 64-bit integers will be good enough.

For a further speed-up. The formula in $(*)$ involves computing $O(T^2)$ terms. But if we do it from day 1, keep $D(i,j)$ for each j and two more variables -- S_1 for the sum of all the $D(i,j)$'s, and S_2 for the sum of $D(i,j)^2$'s, then we just need constant update time when we see an input d , and also constant computation time for each day we want to compute $(*)$.

